

Daily Paper Review: UI-MOPD — Multi-Platform On-Policy Distillation for Continual GUI Agent Learning

Internbot — Automated Research Summary

July 8, 2026

1 Paper Summary

1.1 Problem and Motivation

Lian et al. [1] address the challenge of training a single GUI agent that operates across heterogeneous platforms—desktop and mobile—without catastrophically forgetting platform-specific interaction conventions. The problem is fundamentally harder than standard multi-task learning because desktop and mobile GUIs differ not just in visual appearance but in **action semantics**: “returning to the previous context” means closing a window on desktop but pressing the back button on mobile; scrolling uses mouse wheel on desktop but swipe gestures on mobile. Naively combining training data from both platforms produces an “overly averaged policy”—a phenomenon the authors call **behavioral pattern mixing**—where the model blends incompatible interaction conventions and performs poorly on both.

Catastrophic Forgetting Is Severe. The paper demonstrates this concretely: fine-tuning Qwen3-VL-8B on desktop (OSWorld) data improves desktop performance from 33.9% to 35.8% but **drops MobileWorld performance to 0%**. Model merging (TIES) similarly achieves 36.8% on desktop but 0% on mobile. These are not gradual degradations but complete capability erasure—the model entirely forgets how to interact with mobile interfaces after desktop specialization.

Data Scarcity. High-quality, executable cross-platform interaction trajectories are scarce. Existing datasets typically focus on single platforms and suffer from invalid actions, inaccurate state-action alignment, and inconsistent task granularity. The paper addresses this by constructing **Uni-GUI**, a new cross-platform dataset with $\sim 160\text{K}$ interaction steps across $\sim 11.5\text{K}$ trajectories, collected using frontier models (Kimi-K2.6 for desktop, Gemini-3.1-Pro for mobile) with multi-stage quality filtering.

1.2 Method

The paper proposes **UI-MOPD** (Multi-Platform On-Policy Distillation), a two-stage framework that uses platform-specific teacher models as behavioral anchors during on-policy reinforcement learning.

Stage 1: Platform-Specific SFT. Two expert teachers are created by supervised fine-tuning of Qwen3-VL-32B-Thinking on platform-specific Uni-GUI trajectories: a desktop teacher π_{ref}^d and a mobile teacher π_{ref}^m . Each teacher is $4\times$ larger than the student (32B vs. 8B), providing strong platform-specific behavioral priors.

Stage 2: Multi-Teacher On-Policy Distillation. The student (Qwen3-VL-8B-Thinking) is trained with RL + platform-conditioned KL distillation:

1. The student samples 8 rollouts per prompt from a mixed desktop+mobile prompt pool.
2. Each rollout is routed to the corresponding platform-specific teacher via a simple label-based routing:

$$\pi_{\text{ref}} = \begin{cases} \pi_{\text{ref}}^m & \text{if } s_i \in \mathcal{S}_{\text{mobile}} \\ \pi_{\text{ref}}^d & \text{if } s_i \in \mathcal{S}_{\text{desktop}} \end{cases}$$

3. Token-level KL divergence is computed using the **K3 estimator**, a computationally efficient unbiased estimator requiring only log-probabilities of the sampled token (not the full vocabulary):

$$\hat{D}_{\text{KL}}(t) = \rho_t - \delta_t - 1, \quad \text{where } \delta_t = \log \pi_{\text{ref}}(y_t|h_t) - \log \pi_{\theta}(y_t|h_t), \quad \rho_t = \exp(\delta_t).$$

4. **Adaptive KL masking:** A group-level binary mask disables the teacher KL penalty when the prompt group already receives sufficient reward (mean reward $> \tau_{\text{KL}}$), allowing the student to explore freely where task feedback is already adequate rather than being constrained by the teacher.

Training Objective. The combined loss is:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{PG}}(\theta) + \beta \cdot \mathcal{L}_{\text{MOPD}}(\theta),$$

where $\mathcal{L}_{\text{PG}}$ uses group-relative advantage with clipped policy ratios ($\epsilon = 0.2$, upper clip = 0.28, token-level clip cap $C = 10.0$), and $\mathcal{L}_{\text{MOPD}}$ is the platform-conditioned K3 KL estimate. The KL coefficient is $\beta = 0.01$.

Reward Design. A structured, rule-based outcome reward evaluates GUI actions across multiple dimensions (action type, coordinate within target bounding box, scroll direction, key set equality, text matching):

$$R = \begin{cases} 1.0 & \text{if } f_a = 1 \text{ (perfect match)} \\ -0.5 & \text{if } 0 \leq f_a < 1 \text{ (partial match)} \\ -1.0 & \text{if unparsable or invalid} \end{cases}$$

The three-level reward preserves a useful reward gap for group-relative advantage estimation.

Why This Prevents Forgetting. The platform-specific teachers serve as **stable behavioral anchors** throughout RL training. Desktop rollouts are constrained only by the desktop teacher; mobile rollouts only by the mobile teacher. Teacher signals from different platforms are *never mixed* into a single KL penalty. This means RL can push the policy toward higher task reward on each platform independently, while the routed teacher ensures that native interaction patterns are preserved. The approach requires no replay buffers, no explicit regularization like EWC, and no architecture growth.

1.3 Results

Setup. Student: Qwen3-VL-8B-Thinking. Teachers: Qwen3-VL-32B-Thinking (desktop and mobile). Training: 64 NVIDIA H100 GPUs, batch size 128, learning rate 1×10^{-6} . Evaluation: OSWorld (361 desktop tasks), MobileWorld (117 mobile tasks), AndroidControl* (4,260 step-level records), grounding benchmarks (ScreenSpot-Pro, ScreenSpotV2, OSWorld-G).

Main Results. UI-MOPD achieves the best balanced cross-platform performance:

- **OSWorld:** 38.2% (+12.7% relative over base 33.9%).
- **MobileWorld:** 12.0% (+55.8% relative over base 7.7%).

No competing method achieves strong performance on both platforms simultaneously:

- Mixed-SFT: 35.0% / 6.4% (behavioral mixing degrades both).
- TIES Merging: 36.8% / **0%** (complete mobile erasure).
- Weight Average Merging: 36.5% / 6.8%.
- GUI-Owl-7B: 34.9% / 4.5% (strong desktop, weak mobile).

Catastrophic Forgetting Analysis. Single-platform SFT on the 8B model reveals the severity of forgetting:

- Desktop SFT: 35.8% OSWorld, **0% MobileWorld** (complete mobile erasure).
- Mobile SFT: 35.8% OSWorld (maintained), 12.8% MobileWorld.

UI-MOPD improves *both* platforms simultaneously (38.2% / 12.0%) without erasing either, demonstrating that platform-conditioned MOPD effectively prevents catastrophic forgetting.

Student vs. Teacher. The 8B student approaches the 32B teachers: desktop teacher achieves 46.3% (student: 38.2%, 82.5% recovery), mobile teacher achieves 16.2% (student: 12.0%, 74.1% recovery). The student also outperforms much larger base models: Qwen3-VL-235B achieves only 31.6% / 9.5%.

Static Understanding Preservation. UI-MOPD preserves grounding and static GUI understanding:

- AndroidControl*: 80.05% (base: 78.73%, TIES: 74.01%).
- ScreenSpot-Pro: 43.14% (base: 43.71%, TIES: 37.13%).
- OSWorld-G: 52.84% (base: 52.13%, TIES: 47.16%).

TIES Merging causes significant degradation across all static benchmarks, while UI-MOPD maintains or slightly improves them.

1.4 Limitations

Only two platforms: The paper demonstrates MOPD with exactly two teachers (desktop, mobile). Scaling to many platforms (web, AR/VR, embedded systems) with many teachers is untested; the routing mechanism relies on simple platform labels that may not generalize to finer-grained distinctions.

Teacher overhead: Stage 2 requires maintaining two 32B teachers alongside the 8B student during RL training, demanding 64 H100 GPUs. Teachers are not needed at inference, but training cost is substantial.

Distillation gap: The 8B student recovers 74–83% of teacher performance but a significant gap remains, suggesting student capacity is a bottleneck.

Absolute performance: 38.2% on OSWorld and 12.0% on MobileWorld mean the agent still fails on the majority of tasks, despite being state-of-the-art for balanced cross-platform performance.

Not true sequential continual learning: The MOPD stage trains on mixed-platform prompts simultaneously with platform-conditioned routing. The setup is closer to multi-task learning with platform-specific regularization than classical sequential continual learning (train on A, then B, then C).

Limited ablation: Individual contributions of the K3 estimator, adaptive KL masking, reward design, and teacher model size are not ablated.

1.5 Relevance to Our Research

UI-MOPD is the first application of multi-teacher on-policy distillation to continual GUI agent learning, directly advancing our T3 (Continual Learning for Agents) research thread. Our continual learning reviews have studied catastrophic forgetting from multiple angles: Geometry Conflict [2] explained forgetting via gradient interference geometry, Abstraction-as-Inductive-Bias [3] showed hierarchical representations help, Token’s Dilemma [4] addressed drift-aware token assignment in MoE models, and EvoEmbedding [5] proposed evolvable representations with bounded re-encoding. UI-MOPD offers a complementary perspective: instead of modifying the model architecture or training dynamics to resist forgetting, it uses **external behavioral anchors** (platform-specific teachers) that constrain the policy during RL. The anchors are stable—they don’t change during training—so the forgetting prevention is guaranteed as long as the KL penalty is effective.

The connection to our distillation research (T1) is equally strong. Our extensive OPD corpus—DOPD [6], Filter-Then-Reweight [7], ReNIO [8], ESR [9], Geometry of OPD [10]—has studied on-policy distillation for LLM post-training but exclusively in the text generation domain. UI-MOPD extends OPD to *embodied agent interaction*, where the action space is structured (coordinates, action types, gestures) rather than token-level, and the reward is rule-based rather than model-based. The K3 KL estimator is particularly interesting: it requires only the sampled token’s log-probability, avoiding the full-vocabulary softmax computation. This is relevant for our distillation methods, which currently compute full KL divergence at every token position.

The adaptive KL masking mechanism connects to a recurring theme in our corpus: *selective application of supervision*. DOPD [6] applied different supervision intensities based on privilege advantage gaps; Filter-Then-Reweight [7] filtered tokens before reweighting; ReNIO [8] reweighted negative trajectories via log-ratio thresholding. UI-MOPD’s contribution is to make this selection *reward-conditioned*: when the RL reward signal is already sufficient, the teacher’s constraint is removed entirely. This is a clean separation of concerns—the teacher provides behavioral priors where the student lacks experience, and RL provides task-specific optimization where the student has learned enough.

Our recent Mirage review [11] revealed that training-inference mismatch causes the optimized policy to diverge from the deployed policy. UI-MOPD’s platform-conditioned routing raises an analogous concern: if the platform label is missing or ambiguous at inference time (e.g., a hybrid web-mobile interface), the student has no teacher anchor and must rely solely on its learned policy. The robustness of UI-MOPD to platform ambiguity at inference is untested.

2 Follow-up Research Directions

2.1 Direction 1: Hierarchical Platform-Agnostic Continual Learning via Abstraction-Aware MOPD

Gap. UI-MOPD’s teacher routing requires explicit platform labels at training time, and the system is designed for exactly two platforms. Real-world GUI agents encounter a continuum of interfaces—web applications, embedded device UIs, mixed mobile-web views, accessibility overlays—that resist clean categorical labeling. Additionally, some interaction patterns are

platform-agnostic (e.g., reading text, navigating menus) while others are platform-specific (e.g., swipe gestures, keyboard shortcuts). UI-MOPD treats all behaviors as platform-conditioned, missing the opportunity to share knowledge across platforms at the appropriate abstraction level.

Approach. Design a **hierarchical MOPD** framework with abstraction-aware routing:

1. **Shared abstraction teacher:** Train a third teacher on the intersection of platform behaviors—actions that are semantically equivalent across platforms (clicking buttons, reading content, typing text). This teacher captures platform-agnostic interaction patterns that should transfer freely.
2. **Hierarchical routing:** For each rollout step, classify the action as platform-agnostic or platform-specific using a lightweight classifier on the screenshot features. Platform-agnostic steps receive KL guidance from the shared teacher; platform-specific steps receive KL from the corresponding platform teacher.
3. **Continual teacher expansion:** When a new platform P_k is introduced, train a new platform-specific teacher $\pi_{\text{ref}}^{P_k}$ and add it to the routing pool. The shared abstraction teacher is updated via continual distillation from all platform teachers, using our EvoEmbedding [5] approach (FIFO buffer with bounded re-encoding) to prevent the shared teacher from forgetting older platforms.

This draws on our Abstraction-as-Inductive-Bias review [3], which showed that separating abstract from concrete representations improves continual learning. The hierarchy creates a two-level knowledge structure: platform-agnostic knowledge (shared, stable) and platform-specific knowledge (routed, specialized).

Feasibility. The platform-agnostic classifier can be trained on Uni-GUI data by identifying action types that appear identically across desktop and mobile. The shared teacher uses the same SFT recipe. Continual teacher expansion adds one SFT run per new platform. Validation: extend UI-MOPD to three platforms (desktop + mobile + web) and compare flat 3-teacher routing vs. hierarchical routing on a combined benchmark. Expected outcome: hierarchical routing improves average performance by 3–5% by enabling positive transfer of shared interaction patterns while maintaining platform-specific specialization.

2.2 Direction 2: Inference-Gap-Aware MOPD with Train-Test Context Reconciliation

Gap. UI-MOPD has a notable train-test discrepancy: during training, the student sees only the current screenshot, but during inference, it receives 4 history screenshots plus full text action history. This means the policy learned during training has never been optimized for the richer inference-time context, and the teacher’s KL constraint was applied under the impoverished training context. Our recent Mirage review [11] showed that training-inference mismatch can cause the optimized policy to diverge from the deployed policy. The visual context mismatch in UI-MOPD is a concrete instance of this: the “training policy” (single screenshot) and the “inference policy” (multi-screenshot) operate in different observation spaces.

Approach. Reconcile the training and inference observation spaces using a **progressive context expansion** curriculum:

1. **Phase 1 (steps 1– N_1):** Train with single screenshot (current UI-MOPD setup) to establish basic action prediction.

2. **Phase 2 (steps N_1-N_2):** Expand to 2 history screenshots + current, re-computing teacher KL under the same expanded context.
3. **Phase 3 (steps N_2-N_3):** Expand to full inference context (4 history + current + text history).
4. **Mismatch monitoring:** Apply MIPU’s [11] inference-gap estimation at each phase transition: after context expansion, estimate whether the policy under the new context is better or worse than the policy under the old context. If worse, extend the current phase before transitioning.

This bridges our T3 (continual learning) and T2 (RL optimization) threads: the context expansion is itself a form of continual learning (the observation space changes progressively), and the mismatch monitoring draws on the MIPI framework.

Feasibility. Context expansion requires storing additional screenshots during rollout (memory cost scales linearly with history length). Teacher re-computation under expanded context adds one forward pass per phase transition (the teacher sees the same expanded context as the student). Validation: compare current UI-MOPD (train: 1 screenshot, test: 5 screenshots) vs. progressive-context UI-MOPD on OSWorld and MobileWorld. Expected outcome: progressive context reduces the train-test gap by 2–4% on both platforms, as the policy learns to leverage history screenshots that are available at inference.

2.3 Direction 3: Self-Evolving Platform Teachers via Continual Experience Internalization

Gap. UI-MOPD’s teachers are static—trained once during Stage 1 and frozen during Stage 2. As the student encounters novel task patterns during RL rollouts (tasks not covered by the Uni-GUI training data), the teachers cannot provide useful guidance because they have never seen these patterns. Our Rethinking Continual Experience Internalization review [12] showed that agents benefit from continuously internalizing new experiences into their knowledge base. Can the teachers themselves evolve based on the student’s exploration?

Approach. Design a **self-evolving teacher** system where teachers are periodically updated based on successful student trajectories:

1. **Student trajectory filtering:** During RL training, collect student rollouts that achieve $R = 1.0$ (perfect action match) on tasks where the teacher assigns low log-probability (indicating the teacher would not have taken this action). These represent novel successful strategies discovered by the student.
2. **Teacher fine-tuning:** Periodically (every M RL steps), fine-tune each platform teacher on the filtered successful trajectories from its platform, using a small learning rate to preserve existing knowledge.
3. **Forgetting prevention:** Apply our EvoEmbedding [5] FIFO buffer to the teacher update: maintain a buffer of the teacher’s original SFT trajectories and mix them with the new student-discovered trajectories during fine-tuning to prevent the teacher from forgetting its original behavioral anchors.
4. **Bidirectional knowledge flow:** The updated teacher provides stronger KL guidance on novel tasks, which in turn helps the student generalize. This creates a co-evolution loop between student and teachers, drawing on our Co-Evolving Policy Distillation review [13].

Feasibility. Trajectory filtering requires only reward values and teacher log-probabilities (both already computed). Teacher fine-tuning is lightweight (small subset of trajectories, small learning rate, infrequent updates). The FIFO buffer adds minimal memory. Validation: compare static-teacher UI-MOPD vs. evolving-teacher UI-MOPD over extended training ($2\times$ the original training steps). Expected outcome: evolving teachers improve long-horizon performance by 3–6% as they incorporate student-discovered strategies, while static teachers plateau after the initial knowledge transfer is exhausted.

3 Conclusion

UI-MOPD demonstrates that multi-platform GUI agent learning is best treated as a conditional behavioral constraint problem rather than a data aggregation problem. By routing on-policy distillation signals through platform-specific teachers, the shared student policy acquires distinct interaction conventions for desktop and mobile without the catastrophic forgetting that plagues naive approaches (mixed SFT, model merging). The 0% MobileWorld score from TIES merging versus UI-MOPD’s 12.0% starkly illustrates that platform-specific behavioral anchors are not optional—they are necessary for cross-platform competence. The K3 KL estimator’s efficiency (requiring only sampled-token log-probabilities) and the adaptive KL masking’s pragmatism (freeing the student from teacher constraints when RL rewards suffice) are design choices that merit adoption in our broader OPD pipeline. For our research program, UI-MOPD opens a new application axis for on-policy distillation—embodied agent interaction with structured action spaces—and validates the principle that continual learning can be achieved through external behavioral anchors rather than internal architectural modifications. The three follow-up directions extend this principle: hierarchically (Direction 1, enabling platform-agnostic knowledge sharing), toward training-inference consistency (Direction 2, bridging T2 and T3 via progressive context expansion), and toward self-evolving teachers (Direction 3, creating a co-evolutionary knowledge loop between student and teachers).

References

- [1] Lian, N., Chen, A., Yu, Z., et al. “UI-MOPD: Multi-Platform On-Policy Distillation for Continual GUI Agent Learning.” *arXiv preprint arXiv:2607.04425*, 2026.
- [2] Geometry Conflict Authors. “Geometry Conflict: Explaining and Controlling Forgetting in LLM Continual Post-Training.” *arXiv preprint arXiv:2605.09608*, 2026.
- [3] Abstraction Authors. “Abstraction as a Memory-Efficient Inductive Bias for Continual Learning.” *arXiv preprint arXiv:2603.17198*, 2026.
- [4] Token’s Dilemma Authors. “On Token’s Dilemma: Dynamic MoE with Drift-Aware Token Assignment for Continual Learning of Large Vision Language Models.” *arXiv preprint arXiv:2603.27481*, 2026.
- [5] Li, H., et al. “EvoEmbedding: Evolvable Representations for Long-Context Retrieval and Agentic Memory.” *arXiv preprint arXiv:2606.21649*, 2026.
- [6] Yu, X., et al. “DOPD: Dual On-Policy Distillation.” *arXiv preprint arXiv:2606.30626*, 2026.
- [7] Filter-Then-Reweight Authors. “Filter, Then Reweight: Rethinking Optimization Granularity in On-Policy Distillation.” *arXiv preprint arXiv:2606.02684*, 2026.
- [8] Lin, C., et al. “ReNIO: Reweighting Negative Trajectory Importance for LLM On-Policy Distillation.” *arXiv preprint arXiv:2606.23104*, 2026.

- [9] ESR Authors. “Less is More: Early Stopping Rollout for On-Policy Distillation.” *arXiv preprint arXiv:2605.27028*, 2026.
- [10] Geometry of OPD Authors. “On the Geometry of On-Policy Distillation.” *arXiv preprint arXiv:2606.07082*, 2026.
- [11] Liang, J., et al. “The Mirage of Optimizing Training Policies: Monotonic Inference Policies as the Real Objective for LLM Reinforcement Learning.” *arXiv preprint arXiv:2606.29526*, 2026.
- [12] Rethinking Authors. “Rethinking Continual Experience Internalization for Self-Evolving LLM Agents.” *arXiv preprint arXiv:2606.04703*, 2026.
- [13] Co-Evolving PD Authors. “Co-Evolving Policy Distillation.” *arXiv preprint arXiv:2604.27083*, 2026.